

MediBuddy: System Architecture - Elaborative Description

Technical Design Document

June 2026

Abstract

This document provides a detailed, textually elaborative description of MediBuddy's system architecture. MediBuddy is India's leading full-stack digital primary care platform, serving over 25 million users, 105,000 doctors, 10,800 labs, and 5,300 pharmacies. The architecture is cloud-native, microservices-based, and deployed on AWS. It comprises six logical layers: user experience, API gateway and edge services, core domain services, AI/ML services, data platform, and integration layer. The system is designed for high availability (99.9% uptime), low latency (<200ms API response time), and compliance with NDHM, DPDP Act, and ISO 27001. This document explains each component, its responsibilities, interactions, data flows, and scalability mechanisms in plain text, accompanied by selected diagrams and tables. It serves as a comprehensive reference for engineers, architects, and stakeholders.

Contents

1	Introduction to the System Architecture	4
2	User Experience Layer	4
2.1	Mobile Applications (iOS and Android)	4
2.1.1	Patient App	4
2.1.2	Doctor App	5
2.1.3	Partner Applications	6
2.2	Web Portal and White-label Solutions	6
3	API Gateway and Edge Services	6
4	Core Domain Services	7
4.1	Patient Service	7
4.2	Doctor Service	7
4.3	Appointment and Video Consultation Service	7
4.4	Prescription Service	8
4.5	Lab Integration Service	8
4.6	Pharmacy Integration Service	9
4.7	B2B Corporate Service	9
5	AI/ML Services	9
5.1	AI Symptom Checker	9
5.2	Doctor Matching Algorithm	10
5.3	AI Transcription and Consultation Summary	10
5.4	Prescription Fraud Detection	11
5.5	Predictive Analytics for Chronic Diseases	11
6	Data Platform	11
6.1	Longitudinal EHR (FHIR)	11
6.2	Data Lake and Analytics Warehouse	12
6.3	Real-time Streaming	12
6.4	Caching Layer	12
7	Integration Layer	13
7.1	NDHM/ABDM Integration	13

7.2	Lab Partner Integration	13
7.3	Pharmacy Integration	13
7.4	Insurance API (Cashless Settlement)	14
7.5	Payment Gateway	14
8	Security and Compliance	14
8.1	Data Security	14
8.2	Authentication and Authorization	15
8.3	Compliance Certifications	15
8.4	Audit Logging	15
9	Deployment and Scaling	16
9.1	Cloud Infrastructure	16
9.2	Orchestration and Autoscaling	16
9.3	Disaster Recovery	16
9.4	CI/CD Pipeline	16
10	Data Flow Descriptions (Textual)	17
10.1	Patient Consultation Flow	17
10.2	Lab Order and Report Flow	17
10.3	Employer Analytics Flow	18
11	Performance Metrics and SLAs (Current)	18
12	Cost Optimization Strategies	19
13	Future Architecture Roadmap (2027–2030)	19

1 Introduction to the System Architecture

MediBuddy’s architecture is built on the principles of cloud-native design, microservices, and event-driven communication. The system must handle over 26 million video consultations, 8 million lab tests, and 12 million medicine orders annually, with peak loads during flu seasons or disease outbreaks. Key quality attributes include:

- **High availability:** 99.9% uptime (less than 8.76 hours downtime per year).
- **Low latency:** API response time ≤ 200 ms at the 99th percentile.
- **Video quality:** Connection success rate $\geq 98\%$, median call setup time ≤ 5 seconds.
- **Scalability:** Horizontal scaling to accommodate 2-3x traffic spikes.
- **Security:** End-to-end encryption, role-based access control, audit logging.
- **Compliance:** NDHM (ABDM), DPDP Act, ISO 27001, SOC 2 Type II.

The architecture is divided into six logical layers, each composed of multiple microservices. Communication between services uses synchronous REST APIs (for request-response) and asynchronous Apache Kafka events (for decoupled, reliable message passing). All services run in Docker containers orchestrated by Kubernetes (AWS EKS) across multiple availability zones in the Mumbai region, with a disaster recovery region in Singapore.

2 User Experience Layer

The user experience layer includes all client-facing applications: mobile apps, web portals, and dashboards. These are designed to be responsive, accessible, and offline-capable where necessary.

2.1 Mobile Applications (iOS and Android)

Both the patient and doctor apps are built using React Native, which allows sharing of over 80% of code between platforms while retaining native performance for video and camera features.

2.1.1 Patient App

The patient app is the primary touchpoint for end-users. Key features and their implementation:

- **Video consultation:** Integrated with WebRTC; uses a custom signalling server and Selective Forwarding Unit (SFU) to handle poor network conditions (common in rural India). The app adapts video quality based on bandwidth (720p $\rightarrow$ 480p $\rightarrow$ 240p).

- **E-prescription viewing:** Prescriptions are displayed as PDFs (generated server-side) and also as structured data for accessibility. Users can share prescriptions via WhatsApp or email.
- **Lab test booking:** Integrated with the lab partner network; users see estimated pickup times and can track phlebotomist location on a map.
- **Medicine ordering:** Real-time inventory check, multiple pharmacy options, and delivery tracking.
- **Health records:** Longitudinal record of consultations, prescriptions, lab reports, and immunisations; users can grant temporary access to other doctors.
- **Family accounts:** A single user can manage up to five family members; each member has a separate health record.
- **Wellness tracking:** Steps, sleep, and water intake (optional integration with Google Fit / Apple Health). Gamification includes leaderboards and rewards (discounts on lab tests).
- **Offline mode:** Critical data (past prescriptions, upcoming appointments) are cached using SQLite. Appointments can be booked offline and synchronised when online.

2.1.2 Doctor App

The doctor app is optimised for quick access during consultations:

- **Video call interface:** Doctors see patient information, past history, and AI-suggested triage notes alongside the video feed.
- **EHR access:** With patient consent, doctors can view previous consultations, lab results, and medications. The interface is read-only unless the patient grants write permission.
- **E-prescription writing:** Structured entry with drug-drug interaction checks; digital signature using a PKI certificate issued by the Indian Medical Council.
- **Lab order creation:** A catalogue of LOINC-coded tests; orders are sent directly to the nearest accredited lab.
- **AI-assisted notes:** After the call, a draft consultation summary is generated by the AI transcription service; doctors can edit it before saving.
- **Schedule management:** Integration with Google Calendar; doctors set availability in 15-minute slots.
- **Earnings dashboard:** Real-time view of consultation fees, revenue share, and bonus incentives.

2.1.3 Partner Applications

Separate React Native apps and web portals are provided for labs, pharmacies, and employers.

- **Lab partner app:** Receive lab orders, assign phlebotomists, upload reports (PDF + FHIR Observations), and manage pickup routes.
- **Pharmacy partner app:** Receive medicine orders, confirm stock, update delivery status, and synchronise inventory via API.
- **Employer dashboard (web):** HR teams monitor employee usage, top health issues, and ROI metrics. Real-time charts are powered by GraphQL subscriptions.

2.2 Web Portal and White-label Solutions

The web portal is built with React.js and server-side rendering for SEO. It is used for patients who prefer desktop browsers, for employer analytics (complex dashboards), and for white-label hospital portals. White-label portals are subdomains (e.g., `manipal.medibuddy.com`) that carry the hospital's branding but use MediBuddy's backend. They are deployed as separate React builds with custom themes.

3 API Gateway and Edge Services

All client traffic (mobile, web, partner APIs) first reaches the API Gateway layer. MediBuddy uses Kong Gateway (open-source) running on Kubernetes, with the following responsibilities:

- **Authentication:** JWT validation via Keycloak; OAuth2 tokens for third-party integrations. Doctors and employees use single sign-on (SSO) with their corporate Azure AD or Okta.
- **Rate limiting:** Each user is limited to 100 requests per minute; each partner (lab, pharmacy) to 1000 requests per minute. Exceeding limits returns HTTP 429.
- **Request routing:** Path-based routing to appropriate microservices (e.g., `/api/v1/patients/*` → patient service, `/api/v1/doctors/*` → doctor service).
- **Caching:** Frequently accessed data (doctor profiles, lab test catalog) are cached in Redis (TTL 5-60 minutes) to reduce backend load.
- **Analytics:** Every request is logged to Apache Kafka (topic `api-logs`) for monitoring and security auditing.

The gateway is fronted by AWS CloudFront (CDN) to serve static assets (app bundles, images, videos) with edge caching. For dynamic API requests, CloudFront forwards to an internal Network Load Balancer (NLB) that distributes traffic across Kong replicas.

4 Core Domain Services

The core domain services implement the primary business logic. Each service is a Spring Boot (Java) or Node.js application, with its own database (PostgreSQL or MongoDB) and exposes a REST API.

4.1 Patient Service

Responsible for patient demographics, authentication, consent management, and family linking.

- Stores FHIR Patient resources extended with local fields: `abhaId` (NDHM), `familyRelations`, `employerId`.
- Consent management: Uses FHIR Consent resources to record patient choices (e.g., share lab reports with employer only in aggregated form). The consent engine is enforced at the API level.
- Integration with NDHM Health Locker: When a patient creates an ABHA ID, the service pushes consultation summaries (with consent) to the locker.

4.2 Doctor Service

Manages doctor profiles, credentials, schedules, ratings, and revenue share.

- FHIR Practitioner resource extended with custom fields: `specialty`, `languages`, `consultationFee`, `revenueSharePercent`.
- Schedule management: Integrates with Google Calendar API (doctors link their calendar). Real-time availability is cached in Redis (TTL 30 seconds).
- Rating system: Patients rate doctors after each consultation (1-5 stars). The service computes an aggregated NPS score displayed to patients.

4.3 Appointment and Video Consultation Service

This service handles the end-to-end flow of booking and conducting video consultations.

- **Booking flow:** Patient receives doctor recommendations from the AI matching service, selects a time slot, and the service creates an appointment record (FHIR Appointment resource). A Kafka event `appointment.booked` is published.
- **Video session management:** Uses WebRTC with a custom SFU (Selective Forwarding Unit) deployed on AWS EC2 (auto-scaling group). The SFU handles media relaying, recording (with consent), and adaptive bitrate. A fallback to Agora's API is available if the SFU fails.

- **Recording and transcription:** With patient consent, the video is recorded and stored encrypted in S3 (retention 90 days). The audio is sent to the AI transcription service.
- **Waiting room and notifications:** Uses Firebase Cloud Messaging (FCM) to notify patients when the doctor joins; also sends reminders via SMS.

4.4 Prescription Service

Generates and manages e-prescriptions.

- **Prescription creation:** Doctor selects medicines (RxNorm codes), dosage, duration. The service checks for drug-drug interactions using a Drools rule engine with over 15,000 interaction pairs. Prescriptions are stored as FHIR MedicationRequest resources.
- **Digital signature:** Each prescription is digitally signed with the doctor's PKI certificate (licensed by the Indian Medical Council). The signature is embedded in the PDF and also stored as a hash in the database.
- **QR code:** A QR code containing the prescription ID and a tamper-evident signature is printed on the PDF. Pharmacies scan it to verify authenticity.
- **Fraud detection:** Anomaly detection models flag unusual patterns (e.g., same patient getting same controlled substance from multiple doctors).

4.5 Lab Integration Service

Abstracts the integration with 10,800 partner labs.

- **Order creation:** Doctor selects tests from a LOINC-coded catalogue; the service creates a FHIR DiagnosticOrder. The patient's location (pincode) determines which lab partners are eligible (based on NABL accreditation and coverage).
- **API abstraction:** Most labs use HL7 v2.x; a few use FHIR R4; some legacy labs require CSV file uploads. The service normalises all into internal FHIR model.
- **Sample pickup:** Once a lab accepts the order, the service tracks the phlebotomist's GPS location (via lab app) and provides real-time updates to the patient.
- **Report delivery:** The lab uploads a PDF report plus structured data (FHIR Observation). The service stores both, triggers notifications to patient and doctor, and runs an automated rule engine to flag critical abnormalities.

4.6 Pharmacy Integration Service

Manages medicine ordering and fulfillment.

- **Inventory sync:** Pharmacies push their inventory deltas every 5 minutes (via API). The service caches availability in Redis (TTL 5 minutes). For quick-commerce partners (Zepto), the cache TTL is 1 minute.
- **Order fulfillment:** Patient chooses a pharmacy (default: lowest delivery time). The service sends the order via REST API; the pharmacy responds with acceptance or rejection. If rejected, the order is re-routed.
- **Delivery tracking:** Integrates with third-party logistics APIs (Dunzo, Shadowfax, Delhivery). Status updates (confirmed, picked up, out for delivery, delivered) are pushed to the patient via push notifications.
- **Commission calculation:** Standard commission 12-15%; quick-commerce commission 5-8%. Invoices are generated weekly.

4.7 B2B Corporate Service

Handles employer accounts, employee provisioning, and analytics.

- **Employee provisioning:** Employers can upload a CSV of employees or use the REST API (POST /v1/employees/bulk). The service creates user accounts, sends invitation emails/SMS, and links them to the employer contract.
- **SSO integration:** Supports OIDC with Okta, Azure AD, and Google Workspace. Employees log in with their corporate credentials.
- **Analytics dashboard:** Real-time aggregation of consultation data, lab orders, and medicine orders per employee/department. Uses a streaming pipeline from Kafka to Redis (counters) and Redshift (historical).
- **ROI reporting:** A monthly job calculates absenteeism savings (actuarial model based on claims data), presenteeism savings, and insurance claim reductions. A PDF report is generated and emailed to HR.

5 AI/ML Services

MediBuddy employs several AI models to improve efficiency, reduce costs, and enhance patient outcomes.

5.1 AI Symptom Checker

- **Model architecture:** A transformer-based model (DistilBERT) fine-tuned on 12 million de-identified consultation transcripts (2019-2025). It performs multi-label

classification: top 3 possible conditions (from 450 ICD-11 codes) and urgency level (routine, urgent, emergency).

- **Annotation:** The training data was annotated by 500 senior doctors. Each transcript includes doctor’s final diagnosis, urgency, and prescribed actions.
- **Inference:** Hosted on AWS SageMaker with auto-scaling. Average inference latency <200ms. Cost: \$0.002 per inference.
- **Integration:** Patient describes symptoms via free text or selection from list. The model returns predictions; the UI shows self-care advice for routine cases, else offers to book a doctor.
- **Performance:** Top-3 accuracy 86%; urgency classification accuracy 92%; false negative rate for emergencies (e.g., myocardial infarction) is 0.3% (with a safety net: any mention of chest pain, severe headache, or difficulty breathing automatically escalates).
- **Impact:** 28% of users receive self-care advice and do not proceed to a doctor, saving \$2.80 per user in doctor payments.

5.2 Doctor Matching Algorithm

- **Approach:** Multi-armed bandit with contextual features. The model learns optimal assignment weights through reinforcement learning, balancing patient satisfaction and doctor workload.
- **Features (15):** doctor availability (real-time), patient’s past doctor preference (weight 30%), language match (15%), specialty match (25%), doctor’s NPS rating (10%), current patient volume (10%), time zone proximity (5%), and response time history (5%).
- **Execution:** For each appointment request, the algorithm computes a score for all eligible doctors (same specialty, language, available slot) in real-time (under 100ms). The top 3 are presented to the patient.
- **Outcome:** Average wait time for general physician: 14 minutes; specialist: 6 hours (down from 14 hours before AI). Doctor utilisation increased from 50% to 68%.

5.3 AI Transcription and Consultation Summary

- **Model:** Whisper (OpenAI) fine-tuned on 100,000 hours of medical conversations (doctor-patient). The fine-tuning improves recognition of medical terminology (drug names, anatomy, lab values).
- **Pipeline:** After a video call (with patient consent), the audio track is extracted. Whisper produces a transcript, which is then summarised using a fine-tuned GPT-3.5 model into structured fields: chief complaint, history, examination findings (if any), diagnosis, prescriptions, follow-up plan.

- **Accuracy:** Word error rate (WER) for medical terms is 15%, which is significantly better than out-of-the-box Whisper (35%). Doctors can edit the summary; average editing time is under 30 seconds.
- **Savings:** Saves doctors about 2 minutes per consultation, allowing them to handle 20% more consults per hour.

5.4 Prescription Fraud Detection

- **Rule engine:** Drools with 500+ handcrafted rules covering drug-drug interactions (15,000 pairs), overdose limits (from standard pharmacopoeias), duplicate therapy (same class from different doctors), and controlled substance flags (as per NDPS Act).
- **Anomaly detection:** Isolation Forest model trained on prescription patterns (patient, doctor, drug, dose, frequency). It flags prescriptions that deviate from the norm: e.g., same patient receiving tramadol from three different doctors in one week, or a doctor prescribing 10x normal dose of a medication.
- **Impact:** 6,200+ potential adverse drug events prevented annually; 1,200+ fraudulent prescriptions blocked (mostly for narcotics and sedatives).

5.5 Predictive Analytics for Chronic Diseases

- **Model:** XGBoost trained on 2 million patient records (demographics, lab results, vitals, lifestyle questionnaire). Predicts 3-year risk for type 2 diabetes (AUC 0.89), hypertension (AUC 0.86), and mental health disorders (AUC 0.78).
- **Integration:** The model runs nightly on the data warehouse (Redshift). At-risk patients are flagged, and their employer (if B2B) receives an anonymised aggregate report. The patient receives a personalised recommendation: “You have a high risk of diabetes. Book a free preventive health checkup.”
- **Outcome:** 34% of prediabetic patients who enrolled in CaaS (Circular-as-a-Service) reverted to normal HbA1c levels after 12 months.

6 Data Platform

The data platform is responsible for storing, processing, and serving all persistent data, including the longitudinal EHR, analytics data warehouse, and real-time streams.

6.1 Longitudinal EHR (FHIR)

- **Storage:** AWS HealthLake, a fully managed FHIR R4 store, is used for the canonical EHR. It is backed by a PostgreSQL database with the FHIR schema for transactional queries. HealthLake provides built-in validation, indexing, and versioning of FHIR resources.

- **Resources stored:** Patient, Practitioner, Condition, Observation (labs), MedicationRequest, DiagnosticReport, Consent, Appointment, Encounter. Each resource has a unique logical ID and version history.
- **Scale:** 25 million patients, 650 million observations, over 10 TB of data.
- **Access patterns:** Real-time read/write via API (patient record retrieval). All writes must pass consent verification. Batch analytics queries are offloaded to Redshift.
- **Consent enforcement:** A policy engine intercepts every API call; it checks FHIR Consent resources to ensure the requesting party (doctor, lab, employer) has appropriate permissions. For example, an employer can only see aggregate data, never individual patient records.

6.2 Data Lake and Analytics Warehouse

- **Raw data lake:** All raw events (consultations, prescriptions, lab results, medicine orders, clickstream) are stored in AWS S3, partitioned by date and domain. Data is stored in Parquet format (columnar, compressed) for efficient querying. S3 lifecycle policies move older data (> 12 months) to Glacier for archival.
- **ETL pipelines:** AWS Glue (PySpark) jobs run hourly to transform raw data into a star schema (fact and dimension tables) and load it into Redshift.
- **Data warehouse:** AWS Redshift (RA3 instances) is used for business intelligence, employer dashboards, and AI model training. Typical queries: “What are the top 10 health issues in the IT department of client X?” response time < 5 seconds.
- **Data models:** Fact tables include consultation_facts (foreign keys to patient, doctor, time), prescription_facts, lab_facts. Dimension tables include patient_dim (anonymised), doctor_dim, time_dim.

6.3 Real-time Streaming

- **Event bus:** Apache Kafka (AWS MSK) with 12 topics (e.g., patient-registration, consultation-started, lab-order-created, medicine-order-fulfilled). Each microservice produces events to relevant topics.
- **Stream processors:** Kafka Streams applications perform real-time aggregations: for example, counting consultations per employer per minute for live dashboards.
- **Connectors:** Kafka Connect sinks to S3 (for data lake) and to Redshift (via JDBC sink) for near-real-time analytics.

6.4 Caching Layer

- **Redis cluster:** 6 nodes (3 primary, 3 replica), 100 GB memory. Caches:

- Doctor availability (TTL 30 seconds) – reduces load on Google Calendar API.
- Lab test catalog (TTL 1 hour) – infrequently changed.
- Patient session data (TTL 24 hours) – for seamless experience.
- API rate limit counters (TTL 1 minute) – distributed rate limiting.
- **Hit rate:** Approximately 85% for doctor availability, 70% for patient profile data.

7 Integration Layer

MediBuddy integrates with external systems through well-defined APIs and protocols.

7.1 NDHM/ABDM Integration

- **ABHA ID creation:** Patients can create an Ayushman Bharat Health Account (ABHA) directly within the MediBuddy app, using Aadhaar or mobile number OTP. The system calls the NDHM Health ID API.
- **Health Locker:** With patient consent, consultation summaries (FHIR Composition resources) are pushed to the Health Locker. This allows patients to share their records with any ABDM-compliant provider.
- **Consent manager:** MediBuddy implements the NDHM consent workflow. Patients can grant temporary access to other doctors or hospitals, and the consent is recorded on the national gateway.

7.2 Lab Partner Integration

- **Protocols:** HL7 v2.x (majority of large labs), FHIR R4 (modern labs), and flat file (CSV) for small labs. The Lab Integration Service abstracts these differences, presenting a unified FHIR interface to other MediBuddy services.
- **Sample flow:** When a lab accepts an order, the service provides a webhook endpoint for status updates. The lab's app posts updates (phlebotomist assigned, sample collected, sample received at lab, report ready). The service translates these to patient-friendly notifications.
- **Error handling:** Orders that fail to be accepted within 30 minutes are automatically re-routed to the next available lab. A dead-letter queue holds failed orders for manual intervention.

7.3 Pharmacy Integration

- **API specification:** MediBuddy provides an OpenAPI 3.0 specification that pharmacies implement. Key endpoints: POST /inventory/sync, POST /orders, PUT /orders/id/status.

- **Inventory sync:** Pharmacies are expected to push full inventory deltas every 5 minutes. For quick-commerce partners, a real-time gRPC stream is used to maintain sub-minute accuracy.
- **Order webhook:** Pharmacies call MediBuddy's webhook when an order status changes. The webhook updates the order database and triggers push notifications to the patient.

7.4 Insurance API (Cashless Settlement)

- **Flow:** When a patient with eligible insurance opts for cashless, the system first calls the insurer's eligibility API (SOAP or REST). If eligible, the patient sees the copay amount and confirms. After service (consultation, lab, medicine), MediBuddy submits a claim (FHIR Claim resource) via the insurer's API.
- **Standards:** FHIR Claim for modern insurers, X12 837 EDI for legacy systems. The Insurance Integration Service translates between these.
- **Partners:** Currently integrated with Navi Health (full stack), Digit Insurance (telemedicine only), and Acko (wellness module). Expansion to 5+ insurers planned by FY28.

7.5 Payment Gateway

- **Providers:** Razorpay (primary for India), Stripe (for international diaspora), Paytm (for UPI mandates).
- **Recurring payments:** B2B subscriptions are invoiced annually (payment via invoice or credit card). Family subscriptions are monthly auto-debit via UPI mandate (Paytm). The payment gateway's webhook notifies the billing service of successful payments or failures.
- **Dispute handling:** Refunds are automated via the gateway API for simple cases; complex disputes are handled by a customer support agent using the gateway's dispute dashboard.

8 Security and Compliance

Security is built into every layer of the architecture.

8.1 Data Security

- **Encryption at rest:** All databases (RDS, Redshift, HealthLake), S3 buckets, and EBS volumes use AES-256 encryption with AWS KMS. Keys are rotated automatically every 90 days.

- **Encryption in transit:** TLS 1.3 is enforced for all API endpoints, WebRTC signalling and media (DTLS), and inter-service communication (Istio mTLS).
- **Data masking:** In non-production environments, PII (name, phone, email, ABHA ID) is replaced with synthetic data. Production logs are scrubbed of sensitive fields.

8.2 Authentication and Authorization

- **Identity provider:** Keycloak (self-hosted on Kubernetes) with LDAP fallback. It supports OAuth2, OIDC, and SAML2 for enterprise SSO.
- **MFA:** Mandatory for all doctors (SMS OTP). Optional for patients (encouraged for those with chronic conditions).
- **Role-based access control (RBAC):** Four roles – Patient (own records only), Doctor (assigned patients plus own schedule), Lab/Pharmacy (orders only), Administrator (system). Permissions are enforced at the API gateway and within each microservice.
- **OAuth2 scopes:** Fine-grained scopes like `patient:read`, `patient:write`, `prescription:write`, `lab:read`. Applications request only the scopes they need.

8.3 Compliance Certifications

Table 1: Security and Compliance Certifications

Certification	Status
ISO 27001 (Information Security Management)	Certified (2025)
SOC 2 Type II (Trust Services Principles)	Certified (2025)
NDHM (ABDM) compliance	Certified (2025)
DPDP Act (India) readiness	Implemented (2026)
HIPAA (for US diaspora services)	Planned (2027)

8.4 Audit Logging

- **API access logs:** Every request (including request headers, user ID, timestamp, response status) is logged to Kafka and stored in S3. Retention period: 7 years for compliance.
- **EHR audit trail:** Each read or write of a FHIR resource is recorded in an immutable audit table (using a hash chain). Patients can request an audit report of who accessed their data.
- **Compliance reporting:** Automated monthly reports are generated for the compliance officer, showing access patterns, failed authentication attempts, and data subject requests.

9 Deployment and Scaling

9.1 Cloud Infrastructure

- **Primary region:** AWS ap-south-1 (Mumbai). Three availability zones (AZs) are used for high availability. All production services run here.
- **Disaster recovery region:** AWS ap-southeast-1 (Singapore). Cross-region replication is configured for RDS (read replica), S3 (cross-region replication), and Redis (global datastore). DR failover is semi-automated (scripted but requires manual approval).
- **CDN:** AWS CloudFront with edge locations in Mumbai, Delhi, Singapore, London, and New York. Static assets (app bundles, images) are cached for 24 hours.

9.2 Orchestration and Autoscaling

- **Kubernetes cluster:** AWS EKS (Elastic Kubernetes Service) with 15 node groups. Stateless services (API gateway, patient service, AI inference) run on spot instances (60% of nodes) for cost savings. Stateful services (databases) run on on-demand instances.
- **Horizontal Pod Autoscaler (HPA):** Scales pods based on CPU (>70%), memory (>80%), and custom metrics (requests per second, Kafka lag). The Cluster Autoscaler adds or removes EC2 nodes accordingly.
- **Database scaling:** PostgreSQL read replicas (3 replicas) handle analytics queries and reporting. Aurora Serverless v2 is used for burstable workloads (e.g., payroll-time reporting). Read replicas are promoted to master in case of AZ failure.

9.3 Disaster Recovery

- **RPO (Recovery Point Objective):** 5 minutes. Achieved by AWS DMS (Database Migration Service) continuous replication to the DR region.
- **RTO (Recovery Time Objective):** 1 hour. The DR runbook includes promoting the DR region's databases to primary, redirecting Route53 DNS, and scaling up Kubernetes clusters.
- **Backups:** Daily snapshots of RDS, Redshift, and Elasticsearch (retention 35 days). S3 versioning is enabled; deleted objects are retained for 90 days.

9.4 CI/CD Pipeline

- **Source code management:** GitLab (self-hosted) with branch protection rules (main and staging branches require approval and passing CI).

- **CI process:** GitLab CI runners build Docker images, run unit tests (JUnit, PyTest), integration tests (Testcontainers), and security scans (SonarQube, Snyk). Images are pushed to Amazon ECR.
- **CD process:** ArgoCD (GitOps) syncs Kubernetes manifests from a Git repository to the EKS clusters. Terraform manages infrastructure (VPC, RDS, S3). Deployment cadence: daily to staging, weekly to production with canary (10

10 Data Flow Descriptions (Textual)

10.1 Patient Consultation Flow

1. Patient opens the app and describes symptoms via text or voice.
2. The request goes through API Gateway to the AI Symptom Checker service, which returns predicted conditions and urgency level.
3. If the condition is routine, the app suggests self-care articles. If not, the app offers to book a doctor.
4. Patient clicks “Consult Doctor”. The Doctor Matching service returns up to three available doctors based on specialty, language, and rating.
5. Patient selects a doctor and a time slot. The Appointment service creates an FHIR Appointment resource and publishes an `appointment.booked` event to Kafka.
6. At the scheduled time, the Video Service establishes a WebRTC connection between patient and doctor. The call is recorded (with consent) and stored encrypted in S3.
7. After the call, the audio is sent to the AI Transcription service, which produces a transcript and a structured summary.
8. The doctor reviews and edits the summary, then writes an e-prescription (digitally signed). The Prescription Service stores it as a FHIR MedicationRequest and generates a PDF with QR code.
9. The prescription is sent to the patient. The patient may order lab tests (via Lab Integration) or medicines (via Pharmacy Integration).
10. All events are streamed to Kafka, which feeds the data lake and employer dashboards.

10.2 Lab Order and Report Flow

1. Doctor creates a lab order from within the EHR, selecting tests from a LOINC-coded catalogue. The Lab Integration Service creates a FHIR DiagnosticOrder.
2. The service determines the nearest NABL-accredited lab based on the patient’s pincode and sends the order via HL7/FHIR.

3. The lab’s system receives the order; a phlebotomist is assigned. The lab app updates status (phlebotomist assigned, on the way, sample collected).
4. The patient receives real-time updates via push notification and can track the phlebotomist on a map.
5. The lab processes the sample, then uploads the report (PDF + FHIR Observations) via a webhook.
6. The Lab Integration Service validates the report, stores it in S3, and creates Observation resources in HealthLake.
7. The patient and doctor are notified. The doctor’s dashboard highlights any critical abnormalities (e.g., HbA1c $\geq 9\%$).

10.3 Employer Analytics Flow

1. Employees of a B2B client use the platform. Each consultation, lab order, and medicine purchase generates Kafka events.
2. A Kafka Streams application aggregates events per employer, department, and employee in near real-time. Aggregates are stored in Redis for low-latency access.
3. When HR logs into the employer dashboard, the frontend queries a GraphQL API (Hasura) that fetches current counters from Redis and historical trends from Redshift.
4. Once a month, a scheduled job runs a Python script that queries Redshift to compute ROI: absenteeism savings (based on a model using absence records and consultation data), presenteeism savings, and insurance claim reductions. The results are compiled into a PDF report and emailed to the HR contact.
5. The HR dashboard also allows exporting data to CSV for internal reporting.

11 Performance Metrics and SLAs (Current)

Table 2: System Performance Targets and Actuals (FY26)

Metric	Target	Actual
API response time (p99)	≤ 200 ms	180 ms
Video connection success rate	$\geq 98\%$	98.2%
Video call setup time (median)	≤ 5 sec	4.8 sec
EHR read latency (p99)	≤ 100 ms	85 ms
Batch analytics query (p99)	≤ 5 sec	4.2 sec
System uptime (excluding planned)	99.9%	99.95%
Disaster recovery RTO	≤ 1 hour	45 min

12 Cost Optimization Strategies

To keep cloud costs manageable while scaling, MediBuddy employs several strategies:

- **Spot instances:** 60% of stateless microservices (API gateway, AI inference, video SFU) run on AWS EC2 spot instances, achieving 70% savings compared to on-demand.
- **Reserved instances:** RDS, Redshift, and ElastiCache clusters are covered by 3-year partial upfront reserved instances, saving 40%.
- **S3 lifecycle policies:** Raw logs and old video recordings are moved to S3 Glacier Deep Archive after 90 days, reducing storage costs by 80%.
- **Video optimization:** Adaptive bitrate reduces average bandwidth usage by 30%. Running the SFU in-house (instead of a third-party like Agora) saves approximately \$0.5M per year.
- **AI inference batching:** For non-real-time models (e.g., chronic risk prediction), predictions are batched and run on spot instances, reducing SageMaker endpoint costs by 60%.

13 Future Architecture Roadmap (2027–2030)

- **2027:** Move to multi-region active-active deployment (Mumbai + Singapore) for lower latency in Southeast Asia. Implement GraphQL federation to unify APIs across domains.
- **2028:** Deploy on-device AI for symptom checker (TensorFlow Lite) to enable offline triage in rural areas. Implement blockchain-based prescription audit trail (Hyperledger Fabric) to eliminate prescription fraud.
- **2029:** Launch fully autonomous AI consult for 20 common conditions (e.g., uncomplicated urinary tract infection, viral upper respiratory infection, mild diarrhoea). Human doctor will review a random 10% of cases and supervise the AI.
- **2030:** Adopt federated learning across partner hospitals: train AI models on their local data without the data ever leaving their premises, improving model generalisation while preserving privacy.